

Departamento de Engenharia Informática

Play4Change: Adaptive Learning Powered by AI

A51620 : Radesh Ilesh Gamanbhai Govind (A51620@alunos.isel.pt)

Relatório para a Unidade Curricular de Projecto e Seminário 2025/26
da Engenharia Informática

Professores : Professor Adjunto Prof. Nuno Datia
Professor Adjunto Prof. António Serrador

Abstract

Play4Change is an adaptive learning platform that combines generative AI, gamification mechanics, and peer review to transform educational content into personalized daily challenges. The platform addresses two critical societal needs: sustainability education aligned with the United Nations Sustainable Development Goals, and digital literacy for the next generation of responsible citizens.

From a technical perspective, the system comprises a cross-platform mobile application built with Kotlin Multiplatform (KMP), Decompose, and the MVI pattern for iOS and Android; a Spring Boot and Kotlin backend using Domain-Driven Design, hexagonal architecture, and Clean Architecture; a shared Kotlin module providing type-safe API contracts across the system boundary; and a TypeScript/React landing page with admin panel. Content generation is powered by an AI pipeline combining Mistral LLM via LangChain4j with Retrieval-Augmented Generation (RAG) and pgvector similarity search. All architectural decisions are documented as 16 Architecture Decision Records.

The system prioritizes sustainable and predictable costs, passwordless authentication (magic link + OAuth 2.0), Redis caching with graceful Redis fallback, full observability via Micrometer and Grafana, and readiness for Kubernetes scaling with explicitly defined migration triggers. Peer review of applied tasks provides cost-free assessment while generating additional learning events for reviewers.

Keywords: Adaptive Learning , Gamification , Generative AI , Kotlin Multiplatform , Sustainability , Clean Architecture , Arrow Either , pgvector

Resumo

Play4Change é uma plataforma de aprendizagem adaptativa que combina inteligência artificial generativa, mecânicas de gamificação e revisão por pares para transformar conteúdo educativo em desafios diários personalizados. A plataforma responde a duas necessidades críticas: a educação para a sustentabilidade alinhada com os Objetivos de Desenvolvimento Sustentável das Nações Unidas, e a literacia digital.

Do ponto de vista técnico, o sistema é constituído por uma aplicação móvel multiplataforma em Kotlin Multiplatform (KMP) com Decompose e o padrão MVI para iOS e Android; um servidor Spring Boot em Kotlin com Design Orientado a Domínio, arquitetura hexagonal e Clean Architecture; um módulo Kotlin partilhado que garante contratos de API type-safe entre cliente e servidor; e uma página de destino em TypeScript/React com painel de administração. A geração de conteúdo é realizada por um pipeline de IA que combina o modelo Mistral (via LangChain4j) com Retrieval-Augmented Generation (RAG) e pesquisa por similaridade vetorial com pgvector. Todas as decisões arquiteturais estão documentadas em 16 Architecture Decision Records.

O sistema privilegia custos sustentáveis e previsíveis, autenticação sem palavra-passe (magic link + OAuth 2.0), cache Redis com degradação silenciosa, observabilidade completa com Micrometer e Grafana, e preparação para Kubernetes com gatilhos de migração explicitamente definidos. A revisão por pares das tarefas aplicadas oferece avaliação sem custo marginal de IA, gerando simultaneamente eventos de aprendizagem para os revisores.

Palavras-chave: Aprendizagem Adaptativa , Gamificação , Inteligência Artificial Generativa , Kotlin Multiplatform , Sustentabilidade , Clean Architecture , Arrow Either , pgvector

Contents

List of Figures	xiii
List of Tables	xv
1 Introduction	1
1.1 Context and Motivation	1
1.2 Problem Statement	1
1.3 Objectives	2
1.4 Supervisors	2
1.5 Report Structure	2
2 Background and State of the Art	5
2.1 E-Learning Platforms	5
2.2 Gamification in Education	5
2.3 Adaptive Learning and Flow State	5
2.4 Generative AI and Retrieval-Augmented Generation	6
2.5 Peer Review as an Assessment Mechanism	6
2.6 Sustainability Education and the UN SDGs	6
2.7 Passwordless Authentication	6
2.8 Cross-Platform Mobile Development	7
2.9 Typed Error Handling: Railway Oriented Programming	7
2.10 Backend Technologies	7
2.11 Architectural Patterns	7
2.11.1 Domain-Driven Design	7
2.11.2 Hexagonal Architecture	8
2.11.3 Clean Architecture	8
2.12 Observability: Metrics and Monitoring	8
2.13 Related Work and Gap Analysis	8

3	Play4Change Platform	9
3.1	Vision and Value Proposition	9
3.2	Core Features	9
3.2.1	AI-Generated Content	9
3.2.2	Adaptive Learning	10
3.2.3	Gamification and Peer Review	10
3.2.4	Sustainability Focus	10
3.3	User Roles	10
3.4	The Four-Step Learning Journey	11
3.5	Landing Page and Administration	11
3.6	Platform Constraints and Design Philosophy	11
4	System Architecture	13
4.1	Overview	13
4.2	Architectural Principles	13
4.2.1	Domain-Driven Design — Bounded Contexts	14
4.2.2	Hexagonal Architecture — Ports and Adapters	14
4.2.3	Clean Architecture Layering	14
4.2.4	Typed Error Handling — Railway Oriented Programming	14
4.3	Shared Kotlin Module	15
4.4	Mobile Client — Kotlin Multiplatform	15
4.4.1	Architecture: MVI with BaseComponent	15
4.4.2	Navigation with Decompose	15
4.4.3	Feature Slices — Domain-Driven Code Organisation	16
4.4.4	Dependency Injection: Koin	16
4.4.5	Data and Network Layer	16
4.4.6	Internationalisation	16
4.5	Backend — Spring Boot with Kotlin	17
4.5.1	Technology Stack	17
4.5.2	API Design	17
4.5.3	Docker and Containerization	17
4.6	AI Content Pipeline	17
4.6.1	Module Boundary: Provider Pattern	17

4.6.2	Batch Generation — Fixed Learning Path	18
4.6.3	On-Demand Generation — Adaptive Branches	18
4.6.4	Graceful Degradation	19
4.6.5	Content Extraction	19
4.7	Security Design	19
4.7.1	Passwordless Authentication — Magic Link + OAuth 2.0	19
4.7.2	Role-Based Access Control	20
4.7.3	Minimal Data Retention	20
4.8	Learning Flow and Day Progression	20
4.8.1	Timezone-Aware Day Index	20
4.8.2	Struggle Detection and Adaptive Branching	20
4.9	Caching Strategy	21
4.10	Observability	21
4.11	Infrastructure and Scalability	21
4.11.1	Deployment: Docker Compose	21
4.11.2	Kubernetes Readiness	22
4.12	CI/CD Pipeline	22
4.13	Testing Strategy	22
5	Implementation and Current State	23
5.1	Development Status	23
5.2	Key Implementation Decisions	23
5.2.1	Type Safety Across the System Boundary	23
5.2.2	Arrow Either — Railway Oriented Programming	23
5.2.3	Passwordless Auth Hardening	23
5.2.4	pgvector for Content Deduplication and Struggle Reuse	24
5.2.5	Peer Review — Three-Verdict Majority	24
5.2.6	Adaptive Day Progression	24
5.3	Encountered Challenges	25
5.3.1	PDFBox 3.x Breaking API Change	25
5.3.2	Mistral Prompt Engineering	25
5.3.3	KMP/iOS Memory Model	25
5.3.4	Known Gaps (Documented in ADR-007)	25
5.4	Project Repository Structure	25

6	Conclusion and Future Work	29
6.1	Summary of Contributions	29
6.2	Current Limitations	30
6.3	Future Work	30
6.3.1	Kubernetes Deployment	30
6.3.2	CDN Integration	30
6.3.3	Gamification Completion	30
6.3.4	Advanced Analytics and Reporting	31
6.3.5	Richer Adaptive Model	31
6.3.6	Internationalization Extension	31
6.3.7	Community and Social Features	31
6.3.8	Apple Sign-In and Additional OAuth Providers	31
6.4	Conclusion	31
	References	33
A	Architecture Diagrams and Supplementary Material	i
A.1	Complete Technology Stack	i
A.2	System Component Diagram	i
A.3	Domain Model	i
A.4	AI Pipeline Sequence Diagram	i
A.5	Database Schema (ERD)	i
A.6	Grafana Dashboard Panels	iii
B	Architecture Decision Records	v
B.1	ADR Registry	v
B.2	ADR-001 — Result Type: Custom Implementation → Arrow Either	vi
B.3	ADR-002 — Communication Architecture: Hybrid REST + gRPC	vii
B.4	ADR-003 — AI Content Generation: Batch vs On-Demand	viii
B.5	ADR-004 — Internationalisation Strategy: Hybrid Server + Client	viii
B.6	ADR-005 — Adaptive Learning Path Architecture	ix
B.7	ADR-006 — AI Agent: gRPC Service → Gradle Module Boundary	ix
B.8	ADR-007 — AI Content Generation Pipeline	x
B.9	ADR-008 — Client Architecture: Decompose + MVI + DDD Feature Slices	x

B.10 ADR-009 — Deployment: Docker Compose over Kubernetes + AWS	xi
B.11 ADR-010 — BaseView Scaffold: Edge-to-Edge Insets + Navigation Drawer	xii
B.12 ADR-011 — Passwordless Authentication: Magic Link + OAuth 2.0	xii
B.13 ADR-012 — Topic Content Pipeline: Async Generation, MinIO, Extraction	xiii
B.14 ADR-013 — Learning Flow: Day Progression, Struggle Detection, Caching	xiii
B.15 ADR-014 — Peer Review: Cost-Free Assessment Through Collective Correction	xiv
B.16 ADR-015 — Observability Strategy: Custom Metrics, Prometheus, Grafana	xv
B.17 ADR-016 — Authentication Hardening: Security Audit and Remediation	xv

List of Figures

4.1	High-level system overview of Play4Change.	13
A.1	Play4Change system component diagram.	i
A.2	Play4Change domain model — bounded contexts and key aggregates.	ii
A.3	AI content pipeline: batch ingestion, pgvector deduplication, adaptive reuse.	ii
A.4	Play4Change relational schema.	ii

List of Tables

2.1	Comparison of Play4Change with related platforms.	8
4.1	Backend technology stack.	17
4.2	Cache key patterns and TTLs.	21
4.3	Custom domain metrics.	21
5.1	Development status by component (April 2026).	27
5.2	Custom Result → Arrow Either migration map.	28
A.1	Complete Play4Change technology stack.	iv
B.1	ADR registry — all decisions recorded during the project.	vi

Chapter 1

Introduction

1.1 Context and Motivation

The world simultaneously faces an education crisis and an engagement crisis. Sustainability topics — climate action, responsible consumption, digital citizenship — are recognized as urgent priorities by the United Nations through the 2030 Agenda for Sustainable Development Goals (SDGs) [25]. Yet traditional educational methods fail to convert knowledge into lasting behavioral change. Passive lectures, static documents, and one-size-fits-all curricula produce low retention and even lower motivation in learners who are bombarded with information but rarely empowered to act on it.

Parallel to this, digital literacy has become a fundamental skill in a world where misinformation spreads rapidly and critical thinking about technology is essential. Educators need accessible tools to build and deliver quality learning experiences without requiring deep technical expertise. Learners need experiences that fit their pace, respect their time, and reward consistent effort.

These two challenges share a common root: the gap between content and engagement. Existing platforms either focus on content delivery without personalization, or on engagement mechanics without educational rigor. Play4Change is designed to close that gap.

1.2 Problem Statement

Current e-learning platforms suffer from well-documented limitations:

- **Low engagement:** Completion rates for Massive Open Online Courses (MOOCs) rarely exceed 10% [9].
- **Lack of personalization:** Courses are delivered at a fixed pace regardless of the learner's prior knowledge or performance.
- **Content bottleneck:** Educators must manually author every quiz, challenge, and task, limiting scalability.
- **Weak feedback loops:** Learners receive little indication of their progress relative to their goals or peers.

- **High infrastructure cost:** Many platforms require expensive cloud resources that scale poorly with the user base.

From a systems engineering perspective, there is also a lack of rigorously architected platforms in the educational technology space. Most edtech solutions are built as monolithic applications with tight coupling, making them difficult to maintain, test, and scale.

1.3 Objectives

Play4Change addresses these problems by pursuing the following primary objectives:

1. Design and implement a **cross-platform mobile application** (iOS and Android) using Kotlin Multiplatform (KMP) with Decompose for lifecycle-aware navigation.
2. Build a **scalable, domain-driven backend** capable of delivering personalized daily challenges at predictable cost.
3. Integrate **generative AI** (Mistral LLM) with Retrieval-Augmented Generation (RAG) and vector search for automated, high-quality content creation from educator-supplied materials.
4. Apply **clean architecture, hexagonal architecture, and Domain-Driven Design** (DDD) principles consistently across all system layers.
5. Ensure the system prioritizes **security** (passwordless authentication, minimal sensitive data retention), **accessibility** (caching, low-bandwidth friendliness), and **future scalability** (Kubernetes-ready infrastructure, CDN-friendly design).
6. Provide a **curated topic ecosystem** aligned with UN Sustainable Development Goals and digital literacy curricula.

1.4 Supervisors

This project is supervised by:

- **Prof. Nuno Datia** — ISEL, Engenharia Informática
- **Prof. António Serrador** — ISEL, Engenharia Informática
- **Prof. Michel Vorenhout** — Ureka / Sustainability domain co-supervisor

1.5 Report Structure

The remainder of this report is organized as follows:

Chapter 2 Background and State of the Art — reviews the relevant literature on e-learning, gamification, adaptive learning, AI-generated education, sustainability curricula, and the architectural patterns employed in the system.

Chapter 3 Play4Change Platform — describes the product vision, core features, user roles, and the four-step learning journey.

Chapter 4 System Architecture — presents the technical architecture, technology stack, component breakdown, and the engineering principles that govern design decisions.

Chapter 5 Implementation — details the current development state, testing strategy, CI/CD pipeline, and known challenges.

Chapter 6 Conclusion and Future Work — summarizes the contributions, reflects on the project status, and outlines planned future work.

Chapter 2

Background and State of the Art

2.1 E-Learning Platforms

The e-learning industry has grown substantially over the past decade. Platforms such as Coursera, edX, Udemy, and Khan Academy have demonstrated that digital education can reach millions of learners globally. However, their core model — video lectures supplemented by multiple-choice assessments — has been repeatedly criticized for low engagement and poor knowledge retention [9].

Duolingo [27] is a notable exception: by applying gamification rigorously to language learning, it achieves significantly higher daily active usage than traditional platforms. Play4Change draws inspiration from this approach while extending it to sustainability and digital literacy topics and adding AI-driven content generation.

2.2 Gamification in Education

Gamification is defined by Deterding et al. [5] as the use of game design elements in non-game contexts. In education, the most effective mechanics include:

- **Daily streaks:** reinforce consistent engagement through loss aversion.
- **Leaderboards:** provide social proof and healthy competition.
- **Achievement badges:** signal mastery and milestone completion.
- **Peer review:** builds critical thinking and collaborative knowledge through distributed evaluation.

Play4Change incorporates all of these mechanics. Gamification must serve learning, not replace it — badges are awarded for genuine comprehension, not mere participation.

2.3 Adaptive Learning and Flow State

Adaptive learning systems adjust the difficulty, pacing, and content of lessons based on each learner's performance [26]. The goal is to maintain learners in the flow state [3] — the optimal cognitive zone between boredom and anxiety.

Play4Change implements adaptive difficulty through a struggle detection mechanism (see Section 4.8.2) that triggers AI-generated remediation branches when a learner fails the same task twice consecutively.

2.4 Generative AI and Retrieval-Augmented Generation

Large Language Models (LLMs) have transformed content generation. However, raw LLM output suffers from hallucination and lack of grounding in specific source material. Retrieval-Augmented Generation (RAG) [14] addresses this by augmenting generation with a retrieval step: documents are embedded into a vector database and semantically similar passages are retrieved at inference time to ground the model's response.

Play4Change uses the **Mistral** LLM [18] for content generation, combined with a **pgvector** index over educator-supplied materials. Mistral is selected for its strong performance-to-cost ratio and self-hostable deployment option. **LangChain4j** [13] provides the JVM integration layer between the Spring Boot server and Mistral, behind a provider-agnostic interface (see Section 4.6).

2.5 Peer Review as an Assessment Mechanism

Peer review is a pedagogically superior assessment mechanism for applied, open-ended tasks. The act of evaluating another learner's work is itself a form of retrieval practice — one of the most evidence-backed learning techniques [22]. Calibrated peer review in higher education has demonstrated that distributed assessment across three independent reviewers produces robust signals even for subjective tasks [20].

Play4Change uses majority-vote peer review (2 of 3 verdicts) for todo-action tasks where learners photograph real-world applications of their learning. This achieves zero marginal AI cost per submission while simultaneously creating learning events for the reviewers.

2.6 Sustainability Education and the UN SDGs

The United Nations' 2030 Agenda defines 17 Sustainable Development Goals [25] covering poverty, health, education, climate action, and more. Despite global commitments, integrating SDG content into daily learning routines remains a challenge. Play4Change curates topics around the SDGs and digital citizenship, enabling educators to contribute materials that the AI automatically structures into SDG-tagged learning paths.

2.7 Passwordless Authentication

Traditional password-based authentication is a leading cause of security breaches. The FIDO2 standard [7] and related approaches (magic links, OTPs, social OAuth) eliminate shared secrets entirely. Play4Change adopts passwordless authentication as a first principle: magic links delivered via email for users without social accounts, and OAuth 2.0 ID token verification (Google, Facebook) for users who prefer social login. No password hashes are stored anywhere in the system (see Section 4.7).

2.8 Cross-Platform Mobile Development

Three major approaches compete for cross-platform mobile development:

- **React Native** (Meta): JavaScript/TypeScript bridge to native components.
- **Flutter** (Google): Dart with a custom rendering engine.
- **Kotlin Multiplatform (KMP)** [11]: shares business logic and data layers across platforms while keeping UI native.

Play4Change uses KMP because it allows the team to share domain logic, network code, serialization, and database queries across iOS and Android, while writing native UI in Jetpack Compose (Android) and SwiftUI (iOS). **Decompose** [4] is used for navigation and lifecycle management within KMP, providing a component-based navigation model that works identically on both platforms without relying on Android-specific ViewModels.

2.9 Typed Error Handling: Railway Oriented Programming

Railway Oriented Programming [28] is a functional design pattern for building pipelines where each step either continues on the happy path or short-circuits with a typed error. It avoids thrown exceptions for expected domain failures — validations, not-found cases, business rule violations — replacing them with explicit return types.

Play4Change uses **Arrow** [1] `Either<E, A>` and the `Raise<E>` DSL for this purpose. The custom error hierarchy (`AppError` and its sealed subclasses) is domain-specific and Arrow-independent; Arrow provides the combinators (`bind()`, `ensure()`, `mapError()`, `zip()`) (see Appendix B, ADR-001).

2.10 Backend Technologies

Spring Boot [23] with Kotlin is the chosen backend framework. Kotlin's null-safety, coroutine support, and seamless interoperability with the Java ecosystem make it an ideal fit. The backend is containerized with Docker for reproducible deployments. **MinIO** [17] provides S3-compatible object storage for educator-uploaded PDF materials, running as a Docker Compose service alongside the application.

2.11 Architectural Patterns

2.11.1 Domain-Driven Design

Domain-Driven Design (DDD) [6] organizes software around the business domain rather than technical concerns. Key DDD building blocks used in Play4Change include aggregates, value objects, domain events, and bounded contexts — each with its own model and repository.

2.11.2 Hexagonal Architecture

The hexagonal architecture [2] separates the application core from its infrastructure. The core defines ports (interfaces), and adapters implement those ports for specific technologies. This enables thorough unit testing of domain logic without any infrastructure.

2.11.3 Clean Architecture

Clean Architecture [15] enforces a strict dependency rule: inner layers know nothing about outer layers. In Play4Change: **Domain** → **Application** → **Infrastructure** → **Presentation**. Domain logic is framework-agnostic and testable in isolation.

2.12 Observability: Metrics and Monitoring

Production systems require visibility into runtime behaviour. **Micrometer** [16] provides a metrics façade that decouples instrumentation from any specific backend. **Prometheus** [19] scrapes Micrometer’s HTTP endpoint and stores time-series data. **Grafana** [8] queries Prometheus with PromQL and renders dashboards. This stack runs entirely within Docker Compose, making the full observability pipeline reproducible from source.

2.13 Related Work and Gap Analysis

Table 2.1 summarizes the key differentiators of Play4Change relative to existing platforms.

Table 2.1: Comparison of Play4Change with related platforms.

Feature	Duolingo	Coursera	Khan Academy	Play4Change
AI content generation	×	×	×	✓
Adaptive difficulty	✓	×	Partial	✓
Daily streak mechanics	✓	×	×	✓
Peer review tasks	×	Partial	×	✓
SDG-focused topics	×	Partial	Partial	✓
Passwordless auth	×	×	×	✓
KMP cross-platform	×	×	×	✓
Open-source stack	×	×	×	✓

Chapter 3

Play4Change Platform

3.1 Vision and Value Proposition

Play4Change transforms passive knowledge into active engagement. The platform makes sustainability education and digital literacy accessible, habit-forming, and impactful — one daily challenge at a time. Its core value proposition rests on three pillars:

1. **Learning that creates real change:** content is curated and AI-enriched so that every challenge connects to real-world behavioral outcomes aligned with the UN SDGs.
2. **Minimum friction, maximum impact:** learners receive one challenge per day — bite-sized and achievable — reducing the activation energy required to engage consistently.
3. **A healthy system for providers:** the platform is designed so that its infrastructure costs scale predictably with the user base, preventing the platform from becoming an operational liability.

3.2 Core Features

3.2.1 AI-Generated Content

Topics are created by educators who upload PDF materials or provide URLs. The AI pipeline (Mistral + RAG) processes this content and automatically generates:

- Multiple-choice and open-ended **quizzes**;
- **Daily challenge** prompts (reflection, action-based, or knowledge tasks);
- **Difficulty variants** for adaptive delivery.

Educators review and curate the generated content through the admin panel before it is made available to learners.

3.2.2 Adaptive Learning

The platform adapts to each learner's pace and performance. After each challenge response, the system evaluates correctness, response time, and streak continuity to place the learner on a difficulty trajectory. Learners performing consistently well receive progressively harder challenges; those struggling receive scaffolded hints and easier variants.

3.2.3 Gamification and Peer Review

The gamification layer includes:

- **Daily streaks:** learners are notified if their streak is at risk, leveraging loss aversion.
- **Leaderboards:** scoped to a topic or global, updated daily.
- **Achievement badges:** awarded for milestones (first challenge, 7-day streak, topic completion, peer review contribution).
- **Peer review tasks:** the final task of each topic (todo-action) requires the learner to photograph a real-world application of their learning. Three enrolled peers evaluate the submission by majority vote (2 of 3 verdicts). Reviewers earn 10 points per completed review — the act of evaluating a peer's work is itself a retrieval practice event [22]. Reviewer identity is never disclosed (anonymity prevents social pressure). After submitting their verdict, reviewers see the community vote breakdown — social feedback without prior anchoring bias.

3.2.4 Sustainability Focus

All topics on the platform are curated around the UN Sustainable Development Goals and digital citizenship. Each topic is tagged to one or more SDGs, allowing learners to explore specific goals (e.g., SDG 13 Climate Action, SDG 4 Quality Education) and track their engagement across the SDG framework.

3.3 User Roles

Play4Change defines three primary user roles:

Administrator (Educator) Can create and manage topics, upload learning materials, review AI-generated content, and publish challenges. Administrators access the platform via the hidden admin panel on the landing page.

Learner Can browse and enroll in topics, receive daily challenges, submit responses, participate in peer review, and track their progress and achievements.

System (AI Agent) The internal AI component that processes educator materials, generates challenge content, and adapts difficulty — not a human user but an active system actor.

3.4 The Four-Step Learning Journey

Play4Change bridges the gap between educational content and real behavioral change through a four-step process:

Step 1 — Educators Create Topics Administrators upload PDF materials or provide URLs pointing to relevant resources. The AI pipeline processes the content, generates a structured learning path with daily challenges, and presents it for educator review and approval.

Step 2 — Learners Enroll and Start Students discover topics through the mobile application, enroll in seconds, and begin receiving one challenge per day. The bite-sized format fits any schedule, requiring as little as five minutes of daily engagement.

Step 3 — Progress Through Challenges Each day brings a new challenge — quiz, reflection prompt, or peer task. The AI monitors performance and adapts difficulty to keep each learner in the flow state. Push notifications maintain streak momentum.

Step 4 — Earn Achievements and Grow Learners complete streaks, earn badges, and climb leaderboards. Community peer reviews build deeper understanding and collaborative knowledge. Progress is visualized through the learner's profile, showing SDG engagement and mastery levels per topic.

3.5 Landing Page and Administration

The public-facing landing page is built with TypeScript and React. It serves as the primary marketing and onboarding surface for the platform. A hidden administration route — accessible only to authenticated administrators — provides the content management interface without exposing it to the general public.

Add screenshot of landing page and admin panel once UI is finalized.

3.6 Platform Constraints and Design Philosophy

Play4Change is designed around the following non-negotiable properties:

- **Sustainable, predictable costs:** infrastructure is sized for actual load, not theoretical peaks. Caching and CDN reduce origin server pressure significantly.
- **Accessibility:** the platform must be usable on low-bandwidth connections. Challenge payloads are small; heavy content (videos, large images) is served via CDN.
- **Security by minimization:** the system avoids storing sensitive information wherever possible. Passwordless authentication eliminates the password database entirely.

- **Growth without penalty:** the architecture is designed so that a 10× increase in users does not require a 10× increase in cost or engineering effort. Kubernetes orchestration (planned) and horizontal scaling are first-class citizens.

Chapter 4

System Architecture

4.1 Overview

The Play4Change system is composed of four primary components (Figure 4.1):

1. **Mobile Client** — Kotlin Multiplatform application for iOS and Android.
2. **Backend Server** — Spring Boot 3 application on Kotlin/JVM, running in Docker.
3. **Shared Kotlin Module** — type-safe API contracts and domain models shared between client and server.
4. **Landing Page & Admin Panel** — TypeScript/React web application.

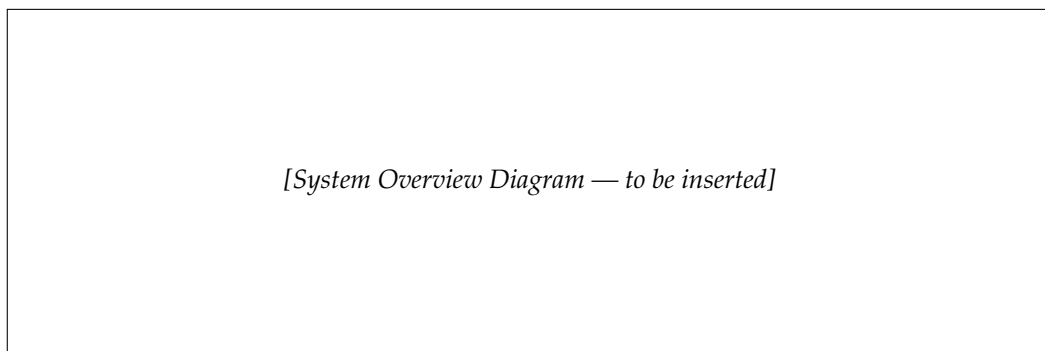


Figure 4.1: High-level system overview of Play4Change.

All architectural decisions are documented as Architecture Decision Records (ADRs) in Appendix B. This chapter presents the resulting architecture; the ADRs document the reasoning and alternatives considered.

4.2 Architectural Principles

Every component follows the same three-layer architectural philosophy.

4.2.1 Domain-Driven Design — Bounded Contexts

The domain is decomposed into five bounded contexts, each with its own aggregate roots, value objects, repositories, and domain events:

- **Identity:** user accounts, passwordless authentication, session tokens, roles.
- **Content:** topics, educator materials, AI-generated task templates.
- **Learning:** enrollments, daily challenge delivery, adaptive difficulty, day progression.
- **Gamification:** streaks, badges, leaderboards, experience points.
- **Peer Review:** open-ended task submission, reviewer assignment, verdict aggregation.

Bounded contexts communicate through domain events where possible, ensuring loose coupling.

4.2.2 Hexagonal Architecture — Ports and Adapters

Each bounded context defines input ports (use case interfaces) and output ports (repository and external service interfaces). Adapters implement output ports for concrete technologies: JPA for relational persistence, Redis for caching, an HTTP adapter for Mistral AI, MinIO for file storage, and Resend for transactional email. Spring annotations, JPA annotations, and external SDK calls live exclusively in infrastructure adapters — the domain and application layers contain no framework imports.

4.2.3 Clean Architecture Layering

The dependency rule is strictly enforced in the server codebase:

1. **Domain:** entities, value objects, domain events, domain services — zero external dependencies.
2. **Application:** use cases, command/query handlers — depends only on domain and port interfaces.
3. **Infrastructure:** JPA adapters, MinIO adapter, Mistral adapter, Redis adapter, email adapter.
4. **Presentation:** REST controllers, request/response DTOs — depends only on application use cases.

4.2.4 Typed Error Handling — Railway Oriented Programming

All use case and service methods return `Either<AppError, T>` from the **Arrow** library [1] rather than throwing exceptions for expected domain failures. The `Raise<E>` DSL enables Railway Oriented Programming [28]: each step either propagates on the right track (success) or short-circuits on the left (typed error). The `AppError` sealed class hierarchy and its HTTP status mappings are domain-specific; Arrow provides the combinators. REST controllers call `.getOrThrow()` which bridges to Spring's `GlobalExceptionHandler`. See Appendix B, ADR-001 for full rationale.

4.3 Shared Kotlin Module

The shared module is a Kotlin Multiplatform library compiled for both the JVM (server) and Kotlin/Native (iOS client). It contains:

- **API contracts:** `@Serializable` Kotlin data classes for all HTTP request and response bodies. Both Ktor (mobile) and Spring Boot (server) use the same classes — a schema change that is not reflected in the shared module produces a compile-time error.
- **Domain value objects:** typed identifiers, enumerations, and shared constraints.
- **Validation logic:** field-level rules that execute identically on client and server.

This module eliminates an entire class of client-server integration bugs. It is the primary mechanism for type safety across the system boundary.

4.4 Mobile Client — Kotlin Multiplatform

4.4.1 Architecture: MVI with BaseComponent

The client follows a three-layer architecture per feature slice (see Section 4.4.3). State management uses the **Model-View-Intent (MVI)** pattern implemented through a `BaseComponent<S, E>` base class (ADR-008):

- **State** (`S : ComponentState`): an immutable data class held in `Decompose`'s `MutableValue<S>`, which integrates with the component lifecycle directly without requiring a coroutine collector.
- **Events** (`E : ComponentEvents`): a sealed class representing all possible user interactions. Screens are pure composables that receive `(state, onEvent)` — no logic lives in UI code.
- **Effects:** one-time navigation or toast signals emitted via a `Channel<Effect>` and collected with `LaunchedEffect`. Navigation is always an effect, never a state field — this prevents re-triggering on recomposition.

4.4.2 Navigation with Decompose

Navigation and lifecycle management use **Decompose** [4]. Each screen is a component with its own lifecycle and back stack managed by `ChildStack`. `DefaultRootComponent` is the single point where `navigation.push()` and `navigation.replaceAll()` are called — composables never invoke navigation APIs directly.

Key navigation invariants:

- `navigateToHome()` and `navigateToLogin()` use `replaceAll` — the back stack is cleared and cannot loop back to a previous auth flow.
- Modal screens use `push` with a guard against duplicate stack entries.

- Transition direction is derived from stack size comparison, not a global flag.

Decompose's `@Serializable Config` sealed interface means the back stack survives Android process death and is restored automatically.

4.4.3 Feature Slices — Domain-Driven Code Organisation

Every feature is a self-contained vertical slice under `features/<feature>/`:

```
features/<feature>/
  domain/model/      -- pure Kotlin data classes
  domain/repository/ -- interfaces only
  data/mock/         -- deterministic fake implementations
  presentation/      -- State, Events, Effect, Component interface
                      DefaultXxxComponent (extends BaseComponent)
  ui/                -- @Composable only (state, onEvent) -- no logic
```

Features never import from each other. Shared types live in `core/`. Production repository implementations in `data/network/` replace mock bindings via a one-line Koin configuration change — zero changes to screens, components, or domain models.

4.4.4 Dependency Injection: Koin

Koin [10] is the DI framework. Hilt and Dagger are excluded because `kapt` (annotation processing) is not available on Kotlin/Native (iOS target). Koin's runtime verification risk is mitigated by integration tests that boot the full Koin module graph and verify all bindings resolve.

4.4.5 Data and Network Layer

- **Ktor** [12]: HTTP client for all API communication (shared in `commonMain`).
- **SQLDelight** [24]: type-safe, KMP-compatible local persistence for challenge caching and offline access.
- **Kotlinx.serialization**: JSON marshaling using the same classes as the shared module.

4.4.6 Internationalisation

All UI strings use `stringResource(Res.string.key)` via the Compose Multiplatform resources system. Hardcoded strings are treated as build violations. Resource files in `commonMain/composeResources/` cover English (default), Portuguese, and French. API error messages are translated server-side via Spring `MessageSource` using the `Accept-Language` header — no per-platform duplication of error strings (ADR-004).

4.5 Backend — Spring Boot with Kotlin

4.5.1 Technology Stack

Table 4.1: Backend technology stack.

Component	Technology
Language	Kotlin (JVM 21)
Framework	Spring Boot 3.x
HTTP API	Spring Web MVC (REST/JSON)
Persistence	Spring Data JPA + PostgreSQL
Vector Search	pgvector (PostgreSQL extension)
Error handling	<code>Arrow Either<AppError, T></code>
Caching	Redis (Spring Data Redis + Lettuce)
File storage	MinIO (S3-compatible, Docker Compose service)
Email	Resend REST API (magic link delivery)
AI integration	LangChain4j + Mistral API
Containerization	Docker / Docker Compose
Observability	Micrometer + Prometheus + Grafana
Auth	Passwordless (magic link + OAuth 2.0)

4.5.2 API Design

The backend exposes a versioned REST JSON API (`/api/v1/...`) consumed by both the mobile client and the admin panel. All endpoints are documented with OpenAPI (Springdoc). API versioning and all endpoint definitions are documented in Appendix A.

4.5.3 Docker and Containerization

A `docker-compose.yml` orchestrates the full environment: the Spring Boot application, PostgreSQL with pgvector extension, Redis, MinIO, Prometheus, and Grafana. This ensures environment parity between development, CI, and production, and means the entire platform — including observability — starts with a single `docker compose up -d`.

4.6 AI Content Pipeline

4.6.1 Module Boundary: Provider Pattern

The AI integration is split across two Gradle modules to decouple the server from any specific LLM provider (ADR-006):

- `:ai-agent:api` — interfaces and data models only; no LangChain4j.
- `:ai-agent:langchain` — LangChain4j + Mistral implementation.

- `:server` — depends only on `:ai-agent:api`.

The `TaskGenerationPort` interface is the contract:

```
interface TaskGenerationPort {
    suspend fun generateTasks(request: GenerationRequest)
        : Either<AppError, GenerationResult>
    suspend fun generateAdaptiveBranch(context: StruggleContext)
        : Either<AppError, AdaptiveBranch>
    suspend fun healthCheck(): Boolean
}
```

Switching from Mistral to another provider requires only a new module implementing this interface and one Spring bean swap — zero changes in `:server`.

4.6.2 Batch Generation — Fixed Learning Path

Fixed-path tasks are pre-generated nightly (ADR-003). A Spring `@Scheduled(cron = "0 0 2 * * *")` job generates tasks for all active topics. Key design properties:

- **Prompt architecture:** two static objects, `TaskGenerationPrompt.system()` and `TaskGenerationPrompt.user()` compose the Mistral prompt. Both are pure Kotlin — independently testable without `LangChain4j`.
- **Resilience:** `@Retryable` (3 attempts, exponential back-off) and `Resilience4j` circuit breaker wrap LLM calls; a static fallback task library is always available.
- **Deduplication:** after generation, each task is embedded and compared against existing embeddings using `pgvector`'s cosine distance operator (`<=>`). Tasks above a 0.90 similarity threshold are discarded to prevent content fatigue.
- **Cost curve:** batch generation scales with domains, not users. At 10,000 users across 5 topics, 50 LLM calls are made per night.

4.6.3 On-Demand Generation — Adaptive Branches

When a learner's struggle is detected (Section 4.8.2), an adaptive branch is generated on-demand. Before calling Mistral, the system queries `pgvector` for similar past struggle embeddings:

- Similarity > 0.90 → reuse existing branch entirely (zero LLM cost).
- Similarity $\in (0.65, 0.90]$ → merge existing tasks with a generated complement.
- Similarity < 0.65 → generate fresh branch; store embedding for future reuse.

This means the adaptive generation cost decreases over time as the pattern library grows. The prompt strategy uses `StruggleAnalysisPrompt` — an “adaptive learning coach” persona rather than a content designer — directing the model toward scaffolded remediation based on the classified error pattern.

4.6.4 Graceful Degradation

Three layers ensure the application runs without an active LLM:

1. `LangChain4jConfig` is `@ConditionalOnExpression` — if `ai.mistral.api-key` is absent, the entire `LangChain4j` bean graph is not loaded.
2. `NoOpTaskGenerationAdapter` activates automatically (`@ConditionalOnMissingBean`) and returns `ServiceUnavailable` for every call.
3. `CourseService` folds on `Either.Left` from the AI port and substitutes pre-authored mock tasks, keeping course creation functional. Mock tasks are replaced on the next successful batch run.

4.6.5 Content Extraction

Educator-supplied materials are processed by `ContentExtractorPort`:

- **PDF:** Apache PDFBox 3.x extracts plain text page by page. Files are stored in MinIO (`topics/{topicId}/source`).
- **URL:** Jsoup fetches the HTML page and strips markup to yield article text (10-second timeout).

Topic creation is fully asynchronous: the HTTP response is returned immediately (status `GENERATING`), and `@Async` drives the extraction $\rightarrow$ embedding $\rightarrow$ generation $\rightarrow$ `ACTIVE` pipeline in a bounded thread pool. A top-level `try/catch` and `withTimeoutOrNull` guarantee the topic always ends in `ACTIVE` or `FAILED` — never stranded in `GENERATING` (ADR-012).

4.7 Security Design

4.7.1 Passwordless Authentication — Magic Link + OAuth 2.0

Play4Change supports two passwordless flows, both converging at a `TokenPair` (access token + refresh token) issued by the Play4Change server (ADR-011):

Magic Link The user provides their email. The server generates a 32-byte cryptographically random token, hashes it with SHA-256, stores only the hash, and emails the raw token in a one-time link via the Resend API [21]. On link click, the server validates the hash, marks it used atomically (single-statement CTE — no TOCTOU race), and issues a `TokenPair`. A `ConsoleEmailAdapter` fallback prints links to stdout in local development.

OAuth 2.0 For Google: the mobile OS authenticates the user with Google and returns a signed ID token. The server verifies the token signature locally using Google’s JWKS endpoint (cached hourly — no per-login network call to Google). The `aud` claim must match the configured `GOOGLE_CLIENT_ID`. For Facebook: the server calls the Graph API `/me` endpoint to verify the access token.

Token Lifecycle Access tokens are JWTs (15-minute expiry, not stored). Refresh tokens are stored as SHA-256 hashes (7-day expiry). On refresh, the old token is marked used and a new pair is issued. If a used refresh token is re-presented, the entire token family is revoked (theft detection). Logout also revokes the full family, terminating all sessions derived from the same login event.

4.7.2 Role-Based Access Control

Two roles: `USER` and `ADMIN`. The role is embedded in every JWT as a claim and propagated through refresh rotation — no database lookup per request. Admin routes are protected by Spring Security (`.requestMatchers("/admin/**").hasRole("ADMIN")`). Admin status is granted by a direct database update — there is no self-service admin registration endpoint by design.

4.7.3 Minimal Data Retention

No password hashes, no payment data, no third-party tracking. Challenge response data is retained only for personalization. Users may request full data deletion (GDPR Article 17).

4.8 Learning Flow and Day Progression

4.8.1 Timezone-Aware Day Index

“One task per day” means calendar days in the user’s local timezone (ADR-013). The `X-Timezone` header (e.g., `Europe/Lisbon`) is parsed on every request. `DayIndexCalculator` uses `ChronoUnit.DAYS.between(LocalDate)` — counting calendar days, not 86,400-second intervals — to compute how many days have elapsed since enrollment. An absent or invalid timezone silently defaults to UTC.

4.8.2 Struggle Detection and Adaptive Branching

The adaptive branch opens after 2 wrong answers on the same task assignment. The threshold is chosen deliberately: 1 wrong answer is normal; 3 means the learner has already spent too long without help. On the second failure, `ErrorPatternClassifier` applies a deterministic 4-rule priority chain (no AI cost) to classify the struggle:

TIME_PRESSURE Answer submitted after the 24-hour task window.

READING_ERROR Selected option is adjacent to the correct answer in the original (unshuffled) order.

PARTIAL_UNDERSTANDING User selected two different wrong options across two attempts.

WRONG_CONCEPT Default — consistent, confident incorrect answer.

The classified pattern drives the adaptive prompt strategy. The adaptive branch is generated asynchronously and inserted into the learner’s roadmap after the current day’s task. After completing all adaptive tasks, the learner resumes the fixed path — no retry of the original task (ADR-005, ADR-013).

4.9 Caching Strategy

Redis is used as a multi-level cache via `CachePort`. All Redis operations are wrapped in try/catch — a Redis outage degrades performance (cache misses) but never breaks correctness; the database remains the source of truth (ADR-013).

Table 4.2: Cache key patterns and TTLs.

What	Key pattern	TTL	Invalidated on
Today's task	<code>task:{userId}:{topicId}:{day}</code>	24h	Submission
Topic metadata	<code>topic:{topicId}</code>	1h	Admin update
Enrollment state	<code>enrollment:{userId}:{topicId}</code>	15m	Any progress update
Task template	<code>template:{id}:{version}</code>	6h	New version created

4.10 Observability

The server is instrumented with **Micrometer** [16] using direct `MeterRegistry` injection (not `@Timed` annotations, which cannot capture domain-specific events). Six custom domain metrics are collected (ADR-015):

Table 4.3: Custom domain metrics.

Metric	Type	Tags
<code>task_generation_duration_seconds</code>	Timer	<code>status</code> (success/failed/timeout)
<code>struggle_sessions_total</code>	Counter	<code>error_pattern</code>
<code>cache_requests_total</code>	Counter	<code>key_prefix</code>
<code>cache_hits_total</code>	Counter	<code>key_prefix</code>
<code>topic_enrollments_total</code>	Counter	—
<code>task_submissions_total</code>	Counter	<code>outcome</code> , <code>task_type</code>
<code>peer_reviews_completed_total</code>	Counter	<code>finalized</code>

Prometheus scrapes `/actuator/prometheus` every 15 seconds. A provisioned Grafana dashboard (9 panels) visualises task generation P95 latency, cache hit ratio by data type, struggle patterns by error category, submission outcomes, JVM heap, and HTTP request rate. All provisioning files are committed to the repository — `docker compose up` delivers a fully instrumented platform with no manual setup.

4.11 Infrastructure and Scalability

4.11.1 Deployment: Docker Compose

The initial deployment uses Docker Compose on a single VPS (ADR-009). Every service maps directly to a future Kubernetes `Deployment` or `StatefulSet`. The nightly batch job maps to a Kubernetes `CronJob`. The `TaskGenerationPort` module boundary is already the correct seam for extracting the AI agent into an independent deployable service when scale demands it.

4.11.2 Kubernetes Readiness

The system is Kubernetes-ready by design:

- All services are stateless; state lives in PostgreSQL, Redis, and MinIO.
- Health endpoints (`/actuator/health`) are exposed for liveness and readiness probes.
- Configuration is externalized via environment variables (12-factor app).
- Horizontal Pod Autoscaling can be applied to API and AI pipeline services independently.

Migration triggers are explicitly defined in ADR-009: resource contention between batch and serving, multi-institution data residency requirements, zero-downtime deployment SLA, or LLM inference becoming a latency bottleneck.

4.12 CI/CD Pipeline

The CI pipeline enforces quality gates on every pull request:

1. Detekt (Kotlin) and ESLint (TypeScript) static analysis.
2. Full test suite (unit, integration via Testcontainers, KMP shared module tests on JVM and Native).
3. Docker image build and tag.
4. Deployment to staging environment on merge to `main`.

4.13 Testing Strategy

The test pyramid is enforced:

- **Unit tests:** domain logic and use cases are tested with plain Kotlin — no mocking of domain objects. `DayIndexCalculator` and `ErrorPatternClassifier` are pure objects testable with literal parameters.
- **Integration tests:** JPA adapters are tested against a real PostgreSQL+pgvector instance via Testcontainers. `PeerReviewService` and `PgVectorDeduplicationService` require real database instances and are covered at this layer.
- **End-to-end tests:** critical user journeys (enrollment, challenge submission, peer review finalization) are tested against the deployed staging environment.
- **KMP tests:** shared module logic runs in both JVM and Kotlin/Native execution environments.

Chapter 5

Implementation and Current State

5.1 Development Status

Play4Change has been implemented across six development phases. Table 5.1 summarizes the current state of each major component.

5.2 Key Implementation Decisions

5.2.1 Type Safety Across the System Boundary

The shared Kotlin module is the most impactful single engineering decision in the project. All HTTP request and response bodies are defined as `@Serializable` Kotlin data classes in `commonMain`. Both the Ktor HTTP client and the Spring Boot controller use these same classes. A change to an API response that is not reflected in the shared module produces a compile-time error, not a runtime crash.

5.2.2 Arrow Either — Railway Oriented Programming

The project began with a custom `Result<T, E>` sealed class (`Result.Success`, `Result.Failure`, `ResultScope`, `resultWrapper`) built intentionally from scratch to internalize the Railway Oriented Programming pattern. After comprehension was established, the codebase was migrated to `Arrow Either<AppError, T>` for production reasons: active maintenance, industry recognition, and rich combinators (`mapError`, `recover`, `zip`). The migration was mechanical (Table 5.2). The `AppError` sealed hierarchy is domain-specific and unchanged. See ADR-001 in Appendix B.

5.2.3 Passwordless Auth Hardening

A post-implementation security audit identified 11 gaps between the ADR-011 design intent and the initial implementation. Key remediations (ADR-016):

- **G1:** Magic link raw tokens were stored in plaintext. Fixed: only `SHA-256(token)` is stored; the raw token travels only in the email link and the single verify request.

- **G2:** The token claim check was not atomic (TOCTOU race condition). Fixed: a single `UPDATE ...WHERE used = false RETURNING *` CTE atomically claims the token.
- **G3:** Google OAuth audience validation was silently skipped when `GOOGLE_CLIENT_ID` was blank. Fixed: blank client ID is a hard startup error; audience is always validated.
- **G5:** Refresh rotation discarded the email claim. Fixed: the `User` entity is loaded by PK during refresh to re-embed the email.
- **G6:** Logout invalidated only the submitted token. Fixed: the entire token family is revoked on logout.
- **G7:** Public routes returned 401 when a stale Bearer token was present. Fixed: `JwtAuthFilter` continues the chain on parse failure for public routes.

5.2.4 pgvector for Content Deduplication and Struggle Reuse

PostgreSQL with the **pgvector** extension serves dual roles: task deduplication in the nightly batch and similarity-based reuse in adaptive branch generation. The cosine distance operator (`<=>`) is accessed via `JdbcTemplate` (not `JPQL`, which does not support custom operators). A custom `VectorConverter` handles bidirectional conversion between Kotlin `FloatArray` and `pgvector`'s string format. Standard CRUD uses JPA; vector queries use `JdbcTemplate` — the right tool for each operation. The decision to use `pgvector` over a dedicated vector database (Qdrant, Weaviate) is documented in ADR-007: at current content volumes, `pgvector`'s performance is equivalent, and it eliminates a deployment component and an additional network hop.

5.2.5 Peer Review — Three-Verdict Majority

The final task of each topic (`TODO_ACTION`) requires the learner to photograph a real-world application of their learning. Three enrolled peers review the submission. Two verdicts (`CORRECT` or `INCORRECT`) trigger finalization synchronously on the third verdict — no async background job needed since finalization is bounded to one assignment update and at most four enrollment point updates. Reviewer selection uses application-level prioritization (unreviewed first, then partial, then random within tier) rather than `ORDER BY RANDOM()` to avoid full-table scans. A unique database constraint (`submission_assignment_id, reviewer_user_id`) provides a secondary safety net against duplicate assignments. After submitting a verdict, reviewers see the percentage breakdown of all verdicts — social feedback without prior anchoring bias (ADR-014).

5.2.6 Adaptive Day Progression

Day index is computed from the `X-Timezone` HTTP header using `ChronoUnit.DAYS.between(LocalDate, LocalDate)` — calendar days, not seconds. The `DayIndexCalculator` is a pure Kotlin object: zero dependencies, zero mocking required in tests. The adaptive roadmap inserts struggle-branch nodes after the current day's task and resumes the fixed path after branch completion — no retry gate, no conditional advancement.

5.3 Encountered Challenges

5.3.1 PDFBox 3.x Breaking API Change

The PDF text extraction library changed its entry point from `PDDocument.load()` to `Loader.loadPDF()` in version 3.x. This was discovered at compile time (correct) rather than runtime (the previous behaviour with a version mismatch).

5.3.2 Mistral Prompt Engineering

Generating consistently structured JSON from an LLM required iterative prompt engineering. The solution combined few-shot examples in the prompt, explicit JSON schema in the system message, defensive markdown-fence stripping in the parser (`removePrefix("`json")`), and `runCatching` per element so malformed tasks skip without aborting the batch.

5.3.3 KMP/iOS Memory Model

Decompose component callbacks and `StateFlow` collectors must be managed through the `Decompose` lifecycle to prevent memory leaks on Kotlin/Native (iOS). The final integration pattern uses `MutableValue` (Decompose-native) rather than `StateFlow`, rendering synchronously on the first composition without a `LaunchedEffect` frame delay.

5.3.4 Known Gaps (Documented in ADR-007)

Two known implementation gaps are tracked:

- Fresh adaptive branch embeddings are not stored after generation, preventing the reuse library from growing from the most expensive generation path. Fix requires adding `struggleEventId` to `StruggleContext` and calling `storeStruggleEmbedding()` after branch persistence.
- `runBlocking` inside `@Transactional` in `CourseService` blocks a thread for the duration of Mistral inference while holding a database connection. Fix requires separating the save and generation transactions.

5.4 Project Repository Structure

The project is organized as a monorepo:

- `/shared` — Shared Kotlin module (KMP): API contracts, value objects, validation.
- `/mobile` — KMP mobile application (`androidApp`, `iosApp`, `commonMain`).
- `/server` — Spring Boot backend.
- `/ai-agent/api` — `TaskGenerationPort` interface and models.

- `/ai-agent/langchain` — LangChain4j + Mistral implementation.
- `/landing` — TypeScript/React landing page and admin panel.
- `/infra` — Docker Compose, Prometheus config, Grafana dashboards, CI/CD.
- `/docs/adr` — Architecture Decision Records (also compiled in Appendix B).

Table 5.1: Development status by component (April 2026).

Component	Status	Notes
Shared Kotlin Module	✓ Done	API contracts, serialization, domain value objects.
Backend — Identity BC	✓ Done	Magic link + Google/Facebook OAuth, JWT, token family revocation, auth hardening.
Backend — Content BC	✓ Done	Topic creation, async generation, PDFBox + Jsoup extraction, MinIO file storage, task versioning.
Backend — Learning BC	✓ Done	Enrollments, day-index progression, struggle detection, adaptive branching, roadmap assembly.
Backend — Gamification	In Progress	Streaks and points implemented; badges and leaderboards pending.
Backend — Peer Review BC	✓ Done	Reviewer selection, 3-verdict majority-vote finalization, social feedback, reviewer rewards.
AI Pipeline (fixed path)	✓ Done	LangChain4j + Mistral, pgvector deduplication, batch @Scheduled, static fallback.
AI Pipeline (adaptive)	✓ Done	Similarity-based reuse (pgvector), StruggleAnalysisPrompt, error pattern classification.
Observability	✓ Done	Micrometer + Prometheus + Grafana, 7 custom domain metrics, provisioned dashboard.
Redis Caching	✓ Done	CachePort with silent fallback, 4 cache domains with TTLs.
KMP Client (Android)	In Progress	Onboarding, auth, topic browser, daily challenge flow, roadmap implemented. Gamification UI pending.
KMP Client (iOS)	In Progress	Shared components compile; SwiftUI integration in progress.
Landing Page + Admin	In Progress	Marketing content live; admin panel for topic management in progress.
CI/CD	Configured	Lint, build, unit tests active; integration tests wired.
Docker Compose	✓ Done	Full stack (server, PostgreSQL+pgvector, Redis, MinIO, Prometheus, Grafana).

Table 5.2: Custom Result $\rightarrow$ Arrow Either migration map.

Custom	Arrow equivalent
<code>Result<T, E></code>	<code>Either<E, T></code>
<code>AppResult<T></code>	<code>Either<AppError, T></code>
<code>Result.Success(data)</code>	<code>Either.Right(data)</code>
<code>Result.Failure(error)</code>	<code>Either.Left(error)</code>
<code>resultWrapper { }</code>	<code>either { }</code>
<code>.bind()</code>	<code>.bind()</code> (same API)
<code>ensure(cond) { err }</code>	<code>ensure(cond) { err }</code> (same)
<code>.fold(onSuccess, onFailure)</code>	<code>.fold(onLeft, onRight)</code>

Chapter 6

Conclusion and Future Work

6.1 Summary of Contributions

This report presents the design and ongoing implementation of Play4Change, an adaptive learning platform powered by generative AI and gamification mechanics. The project has produced the following verified contributions:

1. **A product concept** addressing a genuine gap: combining AI-generated content, adaptive delivery, peer review, and community engagement in a single platform aligned with the UN Sustainable Development Goals.
2. **A cross-platform mobile architecture** using Kotlin Multiplatform, Decompose, and the MVI pattern — sharing all business logic across iOS and Android while preserving native UI on each platform.
3. **A type-safe system boundary** via a shared Kotlin module using `kotlinx.serialization`, eliminating client-server integration bugs at compile time.
4. **A railway-oriented error handling model** using Arrow `Either<AppError, T>`, first built from scratch for comprehension and then migrated to the industry-standard library — demonstrating both understanding and pragmatism.
5. **An AI content pipeline** (LangChain4j + Mistral + pgvector RAG) that reduces the educator’s authoring burden and self-improves through similarity-based reuse of adaptive branch patterns.
6. **A passwordless authentication system** (magic link + OAuth 2.0 with JWKS verification) with security hardening: SHA-256 token storage, atomic CTE claim, token family revocation, and Google audience validation.
7. **A peer review assessment mechanism** using three-verdict majority vote for open-ended tasks — zero marginal AI cost, with pedagogical benefit to reviewers through retrieval practice.
8. **A complete observability stack** (Micrometer + Prometheus + Grafana) with domain-specific metrics — fully provisioned from source, operational from `docker compose up`.
9. **A documented decision trail** of 16 Architecture Decision Records covering every major engineering choice, including alternatives considered and migration paths to future architectures.

10. **A cost-conscious, healthy infrastructure design:** Docker Compose deployment, pgvector over a dedicated vector database, batch AI generation that scales with domains not users, Redis caching with silent fallback, and explicit Kubernetes migration triggers.

6.2 Current Limitations

- **No production deployment:** the system runs in Docker Compose. Production VPS provisioning is pending.
- **iOS UI incomplete:** the SwiftUI layer lags behind Jetpack Compose due to Kotlin/Native interoperability complexity.
- **Gamification partially implemented:** streaks and points are operational; badges and leaderboards are pending.
- **Facebook OAuth not hardened:** audience validation via `/debug_token` requires `FACEBOOK_APP_ID` and `FACEBOOK_APP_SECRET`, not yet configured (ADR-016, G4 deferred).
- **Two known implementation gaps:** fresh adaptive branch embeddings not stored post-generation; `runBlocking` inside `@Transactional` in `CourseService`. Both are documented in ADR-007 with explicit fix descriptions.

6.3 Future Work

6.3.1 Kubernetes Deployment

Migration from Docker Compose to Kubernetes is deferred until one of four explicit triggers is met (ADR-009): resource contention between batch and API serving, multi-institution data residency requirements, zero-downtime deployment SLA, or LLM inference becoming the system bottleneck. The module boundary designed in ADR-006 is already the correct seam for extracting the AI agent into an independently scalable service — zero changes to `:server` are required.

6.3.2 CDN Integration

Static assets (landing page, topic media, educator-uploaded PDFs served for download) will be fronted by a CDN. The provider decision (Cloudflare, AWS CloudFront, BunnyCDN) has not been made; this is a configuration-level change with no application code impact.

6.3.3 Gamification Completion

The badge award system (achievement triggers, badge definitions, profile display) and leaderboard computation (topic-scoped and global, Redis-cached, daily recomputation) are the next implementation priority.

6.3.4 Advanced Analytics and Reporting

An educator-facing analytics dashboard will show cohort engagement, challenge completion rates, SDG topic coverage, and struggle pattern distributions — enabling educators to identify conceptual gaps and improve topic materials.

6.3.5 Richer Adaptive Model

- **Spaced repetition:** schedule challenge reviews based on the learner’s forgetting curve (SM-2 algorithm).
- **Learner interest modeling:** bias content selection toward the SDG tags the learner engages with most.
- **Multimodal challenges:** audio and image-based tasks for learners who respond better to non-textual content.

6.3.6 Internationalization Extension

Current support: English and Portuguese. Extending to French and German is straightforward: add `messages_fr.properties` on the server and `values-fr/strings.xml` on the client. Mistral supports content generation in multiple languages.

6.3.7 Community and Social Features

Group challenges, educator communities, and an open topic marketplace are planned for a future phase once the core learning loop reaches production.

6.3.8 Apple Sign-In and Additional OAuth Providers

Adding new OAuth providers requires one new `OAuthVerifierPort` implementation, one new `AuthProvider` enum value, and one route registration — zero changes to `OAuthService`, `TokenService`, or any domain model (ADR-011).

6.4 Conclusion

Play4Change demonstrates that it is possible to build an educational platform that is simultaneously engaging, architecturally rigorous, cost-sustainable, and secure. The system makes deliberate engineering choices at every layer: type-safe contracts across the client-server boundary, a provider-agnostic AI module that can swap LLMs with a single configuration change, a peer review assessment mechanism that costs nothing at scale while improving reviewer learning, and an observability stack that makes the entire platform’s health visible from day one.

The 16 Architecture Decision Records in Appendix B document not only what was decided but why, what alternatives were considered, and when each decision should be revisited as the system grows.

This decision trail is itself a contribution: it transforms the project from a working implementation into a reproducible, extensible system that future engineers can reason about without re-deriving the original constraints.

The foundation is complete. The remaining work — gamification, iOS UI, production deployment, and the richer adaptive model — will not change the architecture; it will validate it.

References

- [1] Arrow Contributors. “Arrow: Functional companion to kotlin’s standard library”, Accessed: Apr. 1, 2026. [Online]. Available: <https://arrow-kt.io/>
- [2] Alistair Cockburn, “Hexagonal architecture”, *Alistair Cockburn’s Blog*, 2005. [Online]. Available: <https://alistair.cockburn.us/hexagonal-architecture/>
- [3] Mihaly Csikszentmihalyi, *Flow: The Psychology of Optimal Experience*. Harper and Row, 1990.
- [4] Arkadii Kopylov. “Decompose: Lifecycle-aware business logic and navigation”, Accessed: Apr. 1, 2026. [Online]. Available: <https://arkivanov.github.io/Decompose/>
- [5] Sebastian Deterding, Dan Dixon, Rilla Khaled, and Lennart Nacke, “From game design elements to gamefulness: Defining gamification”, in *Proceedings of the 15th International Academic MindTrek Conference*, ACM, 2011, pages 9–15.
- [6] Eric Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003.
- [7] FIDO Alliance. “FIDO2: Web authentication (WebAuthn)”, Accessed: Apr. 1, 2026. [Online]. Available: <https://fidoalliance.org/fido2/>
- [8] Grafana Labs. “Grafana: Operational dashboards”, Accessed: Apr. 1, 2026. [Online]. Available: <https://grafana.com/>
- [9] Katy Jordan, “Initial trends in enrolment and completion of massive open online courses”, *International Review of Research in Open and Distributed Learning*, vol. 15, no. 1, pages 133–160, 2014.
- [10] Kotzilla. “Koin: The kotlin dependency injection framework”, Accessed: Apr. 1, 2026. [Online]. Available: <https://insert-koin.io/>
- [11] JetBrains. “Kotlin multiplatform”, Accessed: Apr. 1, 2026. [Online]. Available: <https://www.jetbrains.com/kotlin-multiplatform/>
- [12] JetBrains. “Ktor: Asynchronous framework for connected applications”, Accessed: Apr. 1, 2026. [Online]. Available: <https://ktor.io/>
- [13] LangChain4j Contributors. “LangChain4j: Java/kotlin llm integration framework”, Accessed: Apr. 1, 2026. [Online]. Available: <https://docs.langchain4j.dev>
- [14] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks”, *Advances in Neural Information Processing Systems*, vol. 33, pages 9459–9474, 2020.
- [15] Robert C. Martin, *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Prentice Hall, 2017.

- [16] Broadcom Inc. “Micrometer: Application observability facade”, Accessed: Apr. 1, 2026. [Online]. Available: <https://micrometer.io/>
- [17] MinIO, Inc. “Minio: High performance object storage”, Accessed: Apr. 1, 2026. [Online]. Available: <https://min.io/>
- [18] Mistral AI. “Mistral ai: Frontier ai in your hands”, Accessed: Apr. 1, 2026. [Online]. Available: <https://mistral.ai/>
- [19] Prometheus Authors. “Prometheus: Monitoring system and time series database”, Accessed: Apr. 1, 2026. [Online]. Available: <https://prometheus.io/>
- [20] Kevin Reily, Patricia Ludford Finnerty, and Loren Terveen, “Two peers are better than one: Aggregating peer reviews for computing assignments is surprisingly accurate”, in *Proceedings of the 2010 ACM conference on Computer Supported Cooperative Work*, ACM, 2010, pages 115–124.
- [21] Resend. “Resend: Email api for developers”, Accessed: Apr. 1, 2026. [Online]. Available: <https://resend.com/>
- [22] Henry L. Roediger and Andrew C. Butler, “The critical role of retrieval practice in long-term retention”, *Trends in Cognitive Sciences*, vol. 15, no. 1, pages 20–27, 2011.
- [23] VMware Inc. “Spring boot reference documentation”, Accessed: Apr. 1, 2026. [Online]. Available: <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/>
- [24] Cash App Contributors. “Sqldelight: Generates typesafe kotlin apis from sql”, Accessed: Apr. 1, 2026. [Online]. Available: <https://cashapp.github.io/sqldelight/>
- [25] United Nations. “Transforming our world: The 2030 agenda for sustainable development”, Accessed: Apr. 1, 2026. [Online]. Available: <https://sdgs.un.org/goals>
- [26] Kurt VanLehn, “The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems”, *Educational Psychologist*, vol. 46, no. 4, pages 197–221, 2011.
- [27] Roumen Vesselinov and John Grego, “Duolingo effectiveness study”, *City University of New York*, vol. 28, pages 1–25, 2012.
- [28] Scott Wlaschin. “Railway oriented programming”, Accessed: Apr. 1, 2026. [Online]. Available: <https://fsharpforfunandprofit.com/rop/>

Appendix A

Architecture Diagrams and Supplementary Material

A.1 Complete Technology Stack

A.2 System Component Diagram

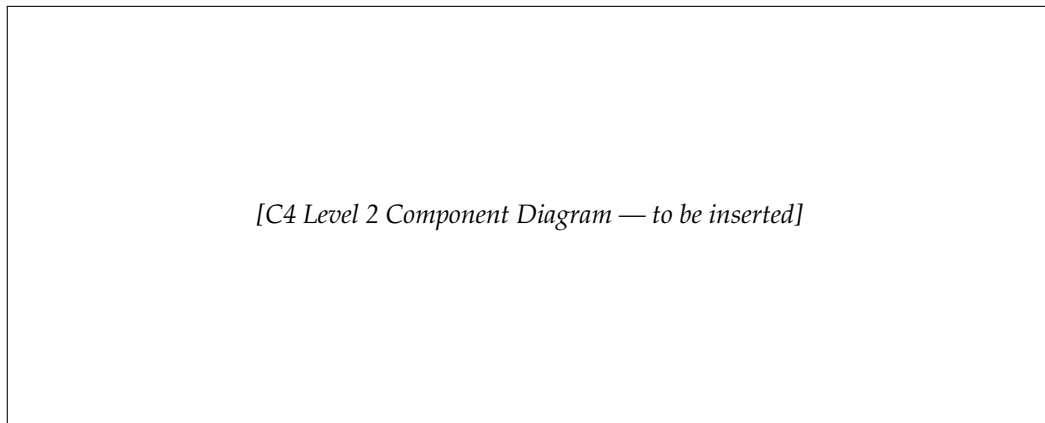


Figure A.1: Play4Change system component diagram.

A.3 Domain Model

A.4 AI Pipeline Sequence Diagram

A.5 Database Schema (ERD)

Key tables: `users`, `magic_link_tokens`, `refresh_tokens`, `topics`, `topic_modules`, `task_templates` (with vector column), `user_enrollments`, `task_assignments`, `struggle_events` (with vector column), `adaptive_branches`, `adaptive_tasks`, `peer_reviews`.

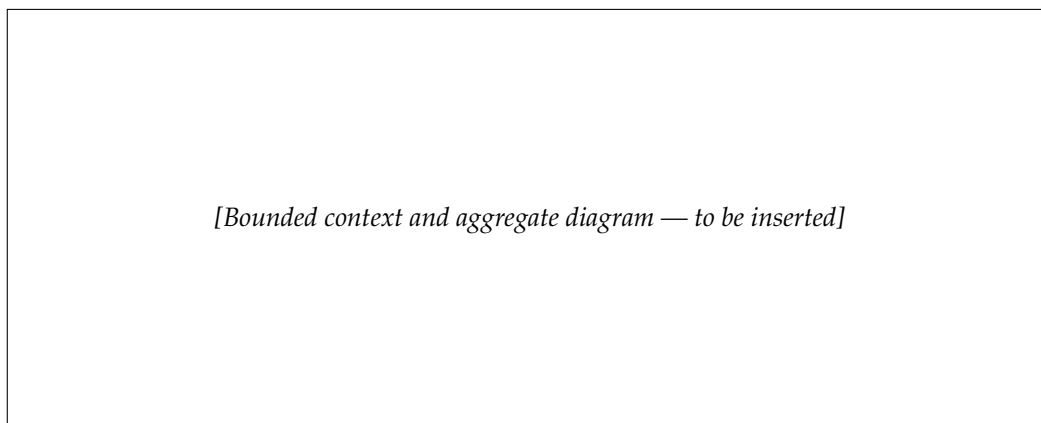


Figure A.2: Play4Change domain model — bounded contexts and key aggregates.

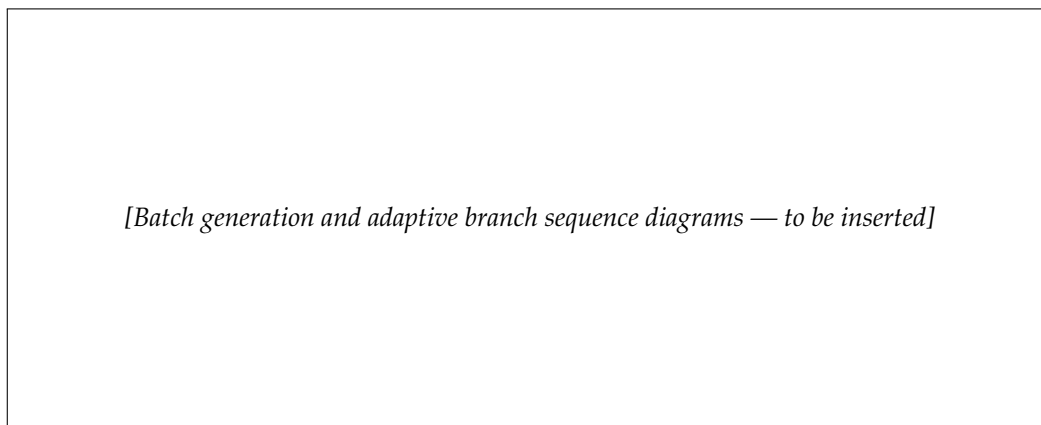


Figure A.3: AI content pipeline: batch ingestion, pgvector deduplication, adaptive reuse.

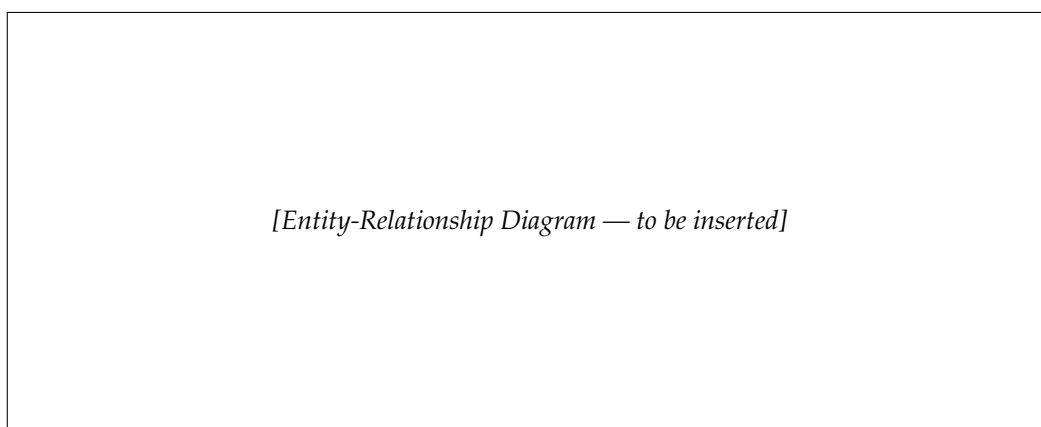


Figure A.4: Play4Change relational schema.

A.6 Grafana Dashboard Panels

The provisioned Grafana dashboard contains 9 panels (ADR-015):

1. Task Generation P95 Duration — `histogram_quantile(0.95, rate(task_generation_duration_seconds))`
2. Task Generation Success Rate — ratio of `status="success"` to `total`
3. Struggle Sessions by Error Pattern — bar chart, 1-hour window
4. Cache Hit Ratio by Key Prefix — `cache_hits_total / cache_requests_total`
5. Enrollments Over Time — hourly increase
6. Task Submission Outcomes — pie chart by `outcome` tag
7. Peer Review Verdict Rate — rate per minute by `finalized` tag
8. JVM Heap Usage — `jvm_memory_used_bytes{area="heap"}`
9. HTTP Request Rate — `rate(http_server_requests_seconds_count[5m])`

Table A.1: Complete Play4Change technology stack.

Layer	Technology	Notes
<i>Mobile Client</i>		
Shared logic	Kotlin Multiplatform (KMP)	Business logic, API contracts
Android UI	Jetpack Compose	Native UI
iOS UI	SwiftUI	Native UI
Navigation	Decompose	Lifecycle-aware, shared
State mgmt	MVI via BaseComponent	Unidirectional data flow
DI	Koin	KMP-compatible, runtime-verified
Local DB	SQLDelight	Type-safe KMP persistence
HTTP Client	Ktor	KMP, shared module
Serialization	Kotlinx.serialization	Shared with server
<i>Backend Server</i>		
Framework	Spring Boot 3.x / Kotlin	JVM 21
Error handling	Arrow Either	Railway Oriented Programming
Database	PostgreSQL + pgvector	Primary store + vector search
Cache	Redis (Lettuce)	Silent fallback on unavailability
File storage	MinIO	S3-compatible, Docker service
Email	Resend REST API	Magic link delivery
AI model	Mistral (via LangChain4j)	Generative AI
Containerization	Docker / Compose	Dev, staging, production
<i>AI Module</i>		
Integration	LangChain4j	JVM-native LLM framework
Port API	TaskGenerationPort	Provider-agnostic interface
Resilience	Resilience4j circuit breaker	Protects against LLM unavailability
<i>Infrastructure</i>		
Orchestration (planned)	Kubernetes	Production scaling
CDN (planned)	TBD	Static assets, media
CI/CD	GitHub Actions (planned)	Lint, test, build, deploy
Auth	Passwordless (magic link + OAuth)	No password hashes stored
<i>Observability</i>		
Metrics	Micrometer	Provider-agnostic façade
Scraping	Prometheus	Time-series database
Dashboards	Grafana	Provisioned from source
<i>Web Frontend</i>		
Language	TypeScript	Landing page, admin panel
Framework	React	

Appendix B

Architecture Decision Records

Architecture Decision Records (ADRs) document the significant engineering decisions made during the development of Play4Change — the context, the decision, the alternatives considered, and the consequences. They serve as an audit trail of the reasoning behind the system’s current design and as a guide for future evolution.

B.1 ADR Registry

The following sections present each ADR in condensed form — context, decision, and key rationale. Full ADR documents (including all alternatives considered and consequences) are maintained in `docs/adr/` in the project repository.

Table B.1: ADR registry — all decisions recorded during the project.

ID	Title	Status	Phase
ADR-001	Result Type: Custom Impl → Arrow Either	Accepted	1 — Core
ADR-002	Communication: Hybrid REST + gRPC	Superseded by ADR-006	1 — Core
ADR-003	AI Content: Batch vs On-Demand	Accepted (Hybrid)	2 — AI
ADR-004	Internationalisation: Hybrid Server + Client	Accepted	1 — Core
ADR-005	Adaptive Learning Path Architecture	Accepted	2 — AI
ADR-006	AI Agent: gRPC → Gradle Module Boundary	Accepted	2 — AI
ADR-007	AI Content Generation Pipeline	Accepted	2 — AI
ADR-008	Client: Decompose + MVI + DDD Feature Slices	Accepted	3 — Client
ADR-009	Deployment: Docker Compose over Kubernetes	Accepted	3 — Client
ADR-010	BaseView Scaffold: Edge-to-Edge + Drawer	Accepted	3 — Client
ADR-011	Passwordless Auth: Magic Link + OAuth 2.0	Accepted	4 — Learning
ADR-012	Topic Pipeline: Async + MinIO + Extraction	Accepted	4 — Learning
ADR-013	Learning Flow: Day Progression + Caching	Accepted	4 — Learning
ADR-014	Peer Review: Cost-Free Collective Assessment	Accepted	5 — Peer Review
ADR-015	Observability: Metrics + Prometheus + Grafana	Accepted	6 — Observability
ADR-016	Auth Hardening: Security Audit + Remediation	Accepted	6 — Observability

B.2 ADR-001 — Result Type: Custom Implementation → Arrow Either

Status: Accepted **Date:** March 2026

Context

The application requires type-safe error handling that avoids exceptions for expected domain failures. The goal is Railway Oriented Programming [28] — a pipeline where each step either continues on the happy path or short-circuits with a typed error.

Decision

Phase 1 — built a custom `Result<T, E>` sealed class with `ResultScope`, `bind()`, `ensure()`, `ensureNotNull()`, and coroutine-safe wrappers. This was intentional: building the pattern from scratch to achieve deep comprehension before adopting a library.

Phase 2 — migrated to Arrow [1] `Either<E, A>` and the `Raise<E>` DSL. The custom `AppError` interface and sealed error hierarchy are retained — they are domain-specific and Arrow has no opinion on them. The `bind()`, `ensure()`, and `ensureNotNull()` APIs are identical, making the migration mechanical (see Table 5.2).

Rationale

Building custom proved comprehension. Shipping with Arrow proves pragmatism. Both matter; the sequence matters. Arrow is actively maintained, industry-standard among Kotlin engineers, and provides combinators (`mapError`, `recover`, `zip`) the team would otherwise have to build.

B.3 ADR-002 — Communication Architecture: Hybrid REST + gRPC

Status: Superseded by ADR-006 **Date:** March 2026

Context

Original design: REST for client-to-server communication; gRPC for server-to-AI-agent communication (AI was planned as a separate Cloudflare Workers TypeScript service).

Decision (original)

REST for the mobile client (cacheable, stateless, KMP/Ktor native); gRPC for the AI agent boundary (typed protobuf contracts, binary-efficient, streaming).

Why Superseded

The AI stack changed from Cloudflare Workers (TypeScript) to LangChain4j + Mistral running on the same JVM. With both ends in the same process, gRPC is the wrong abstraction — serialization overhead without any of the benefits. See ADR-006.

Historical Artifact

The proto definition is retained in `docs/proto/task_generation.proto`. The `TaskGenerationPort` interface designed for the gRPC boundary is now the Gradle module boundary — the same interface becomes a gRPC stub when scale demands extraction.

B.4 ADR-003 — AI Content Generation: Batch vs On-Demand

Status: Accepted (Hybrid) **Date:** March 2026

Decision

Fixed-path tasks: nightly batch via `Spring @Scheduled(cron = "0 0 2 * * *")`. Cost scales with domains, not users — at 10,000 users across 5 topics, ≈ 50 LLM calls per night vs $\approx 10,000$ on-demand calls per day.

Adaptive branches: on-demand, triggered by struggle detection. Volume is low (only struggling users) and latency is acceptable (user is paused awaiting help).

Similarity-Based Reuse

Before generating adaptive content, pgvector is queried for similar past struggles. Similarity > 0.90 : full reuse (zero LLM cost). Similarity > 0.65 : partial merge. Below 0.65: fresh generation. This cost curve decreases over time as the pattern library grows.

Resilience

`@Retryable` (3 attempts, exponential back-off), `Resilience4j` circuit breaker, and a static fallback task library ensure generation failures never prevent course creation.

B.5 ADR-004 — Internationalisation Strategy: Hybrid Server + Client

Status: Accepted **Date:** March 2026

Decision

API error messages are translated server-side: `Spring MessageSource` resolves `AppError.messageKey + params + locale` from the `Accept-Language` header. `messages.properties` (EN, default), `messages_pt.properties` (PT), `messages_fr.properties` (FR extended objective).

UI labels are translated client-side: `Compose Multiplatform` resource system (`stringResource(Res.string.key)`), with `commonMain/composeResources/values/strings.xml` and `values-pt/`, `values-fr/`.

Rationale

Server-side for errors = one source of truth across Android, iOS, and any future web client; adding a new language is one `.properties` file with zero app updates. Client-side for UI = works offline, no round trip for static labels, uses the standard mechanism each platform already provides.

B.6 ADR-005 — Adaptive Learning Path Architecture

Status: Accepted **Date:** March 2026

Decision

Fixed path as the spine; adaptive branches as temporary detours. The user always returns to the fixed path after completing the branch — no retry of the original task.

Course Lifecycle

CREATED → SUBSCRIPTION_OPEN → SUBSCRIPTION_CLOSED → IN_PROGRESS → COMPLETED. The course ends when the *last enrolled user* completes it — not on a fixed date. Adaptive branching directly impacts course completion by accelerating struggling learners.

Key Data Entities

`courses`, `course_modules`, `task_templates` (with embedding vector), `user_course_enrollments`, `user_task_progress`, `struggle_events` (with embedding vector), `adaptive_branches`, `adaptive_tasks`.

B.7 ADR-006 — AI Agent: gRPC Service → Gradle Module Boundary

Status: Accepted **Date:** March 2026

Context

When the AI stack changed to LangChain4j on the same JVM, gRPC between two modules in the same process was the wrong abstraction: serialization overhead without benefits.

Decision

Replace gRPC with a Gradle module boundary using the Provider pattern:

```
:ai-agent:api          ← TaskGenerationPort interface only
:ai-agent:langchain ← LangChain4j + Mistral implementation
:server                ← depends on :ai-agent:api only
```

`:server` never imports LangChain4j. Switching LLM providers: create a new `:ai-agent:openai` module, implement `TaskGenerationPort`, change one line in `settings.gradle.kts`. Zero changes in `:server`.

Future Migration Path

If the AI agent must scale independently: extract `:ai-agent:langchain` into a separate service, create `:ai-agent:grpc-client` implementing `TaskGenerationPort` via gRPC. The interface designed today becomes the service contract tomorrow.

B.8 ADR-007 — AI Content Generation Pipeline

Status: Accepted **Date:** March 2026

Decision

Single `TaskGenerationPort` with two operation contracts: `generateTasks()` for fixed-path content, `generateAdaptiveBranch()` for on-demand remediation. Both use Mistral via `LangChain4j` but with separate prompt strategies: `TaskGenerationPrompt` (content designer persona) and `StruggleAnalysisPr` (adaptive learning coach persona).

pgvector as Load-Bearing Infrastructure

pgvector serves both deduplication (batch) and similarity-based reuse (adaptive). At current content volumes, pgvector's performance is equivalent to a dedicated vector database. `PgVectorDeduplicationService` uses `JdbcTemplate` for cosine distance queries (`<=>` operator not supported by JPQL).

Three-Layer Graceful Degradation

1. `@ConditionalOnExpression` on `LangChain4jConfig`: absent API key → bean graph not loaded.
2. `NoOpTaskGenerationAdapter` (`@ConditionalOnMissingBean`): returns `ServiceUnavailable` for every call.
3. `CourseService` mock fallback: pre-authored tasks substitute on `Either.Left` from the AI port.

Known Gaps (documented)

(1) Fresh adaptive branch embeddings not stored → reuse library does not grow from the most expensive path. (2) `runBlocking` inside `@Transactional` blocks a DB connection during Mistral inference.

B.9 ADR-008 — Client Architecture: Decompose + MVI + DDD Feature Slices

Status: Accepted **Date:** March 2026

Decision

Three-layer client architecture: Decompose `ChildStack` for navigation, MVI via `BaseComponent<S, E>` for state management, and domain-driven feature slices for code organisation.

Why `MutableValue` over `StateFlow`

Decompose's `MutableValue` integrates with the component lifecycle — no coroutine collector, no memory leak risk on Kotlin/Native (iOS), renders synchronously on first composition (no `LaunchedEffect` frame delay).

Events vs Effects

Navigation is always an `Effect` (one-time signal via `Channel<Effect>`), never a state field. Putting navigation targets in state risks re-triggering on recomposition — a documented bug class in `ViewModel`-based architectures.

Feature Slice Structure

`domain/model`, `domain/repository` (interfaces), `data/mock` (deterministic fakes with `delay(600)`), `presentation` (`State`, `Events`, `Effect`, `Component`, `DefaultXxxComponent`), `ui` (pure composables). Switching from mock to production: one line in one Koin DI module.

B.10 ADR-009 — Deployment: Docker Compose over Kubernetes + AWS

Status: Accepted **Date:** March 2026

Decision

Deploy on Docker Compose on a single-node VPS. Do not use Kubernetes or AWS managed services at this stage.

Rationale

At demo scale (1–5 concurrent users, 1 active topic), Docker Compose on a €4–8/month VPS provides identical availability to an EKS cluster at 1% of the cost. AWS EKS minimum: €150–250/month (control plane + worker nodes + load balancer), with none of the problems Kubernetes solves present at current scale.

Architecture Remains Kubernetes-Ready

Each Docker Compose service maps directly to a Kubernetes `Deployment`. Configuration is all environment variables. The nightly batch maps to a `CronJob`. The `TaskGenerationPort` module boundary is already the correct seam for future service extraction.

Migration Triggers

Migrate to Kubernetes when: (1) batch jobs measurably increase API P95 latency, (2) multi-institution data residency is required, (3) a zero-downtime SLA is committed, or (4) LLM inference becomes the latency bottleneck.

B.11 ADR-010 — BaseView Scaffold: Edge-to-Edge Insets + Navigation Drawer

Status: Accepted **Date:** March 2026

Context

Two visual defects (double gap at screen edges from unabsorbed window insets; inconsistent per-screen layout handling) and one UX problem (log-out buried at the bottom of the Profile screen).

Decision

Replace `Box + padding(24.dp)` with `Scaffold(contentWindowInsets = WindowInsets.safeDrawing)` as the `BaseView` implementation. Add optional `topBar`, `bottomBar`, `drawerState`, and `drawerContent` parameters. Move log-out to a `ModalNavigationDrawer` on Home, positioned at the bottom of the drawer panel with a visual divider.

Rationale

`enableEdgeToEdge()` instructs the system to draw behind system bars; `Scaffold` with `safeDrawing` absorbs bar heights into `innerPadding`, eliminating the double-gap for all screens simultaneously. Log-out is a session-level action (terminates the entire session) — it belongs with session-level navigation, not nested in profile data.

B.12 ADR-011 — Passwordless Authentication: Magic Link + OAuth 2.0

Status: Accepted **Date:** April 2026

Decision

Two complementary mechanisms converge at the same `TokenPair` output:

Magic Link: `SHA-256(token)` stored (not plaintext); 15-minute expiry; atomic CTE claim prevents TOCTOU race; delivery via Resend REST API (`ConsoleEmailAdapter` fallback for local dev).

OAuth 2.0: Google uses JWKS local verification (1-hour cache — no per-login call to Google's `tokeninfo` endpoint); audience claim validated against `GOOGLE_CLIENT_ID`; Facebook uses Graph API `/me` (correct for opaque tokens).

Tokens: access JWT (15 min, `userId + email + role` claims, not stored); refresh token (7 days, stored as SHA-256 hash); token family rotation with reuse detection (entire family revoked on reuse).

Why not Firebase Authentication

Vendor lock-in (user identity owned by Firebase), loss of custom user model control, cost at scale (\$0.0055/MAU above 10,000), and the educational value of implementing the security model from first principles.

B.13 ADR-012 — Topic Content Pipeline: Async Generation, MinIO, Extraction

Status: Accepted **Date:** April 2026

Decision

Async: `Spring @Async("generationExecutor")` with bounded thread pool (3 threads, 25-item queue). `withTimeoutOrNull(timeoutMs)` wraps the LLM coroutine — zero race window vs a `@Scheduled` cleanup job. Top-level `try/catch` guarantees topics end in `ACTIVE` or `FAILED` — never stranded in `GENERATING`.

File storage: MinIO (S3-compatible Docker service). `FileStoragePort` interface — swapping to AWS S3 or Cloudflare R2 is a three-line `.env` change with zero application code changes.

Extraction: `PDFBox 3.x` for PDF text (note: breaking API change from 2.x — `Loader.loadPDF()` not `PDDocument.load()`); `Jsoup` for URL text (10-second timeout).

Task versioning: editing a task creates a new `TaskTemplate` (`version++`, `isCurrent=true`); old version soft-deleted. In-progress learners keep their version.

B.14 ADR-013 — Learning Flow: Day Progression, Struggle Detection, Caching

Status: Accepted **Date:** April 2026

Day Progression

`DayIndexCalculator: ChronoUnit.DAYS.between(enrolledLocalDate, todayLocalDate)` in the user's timezone from the `X-Timezone` header. Calendar days, not 86,400-second intervals. Invalid or absent timezone silently defaults to UTC. Pure Kotlin object — zero dependencies, zero mocking required.

Struggle Detection

Threshold: 2 wrong answers on the same `TaskAssignment`. `ErrorPatternClassifier`: 4-rule priority chain (`TIME_PRESSURE` → `READING_ERROR` → `PARTIAL_UNDERSTANDING` → `WRONG_CONCEPT`). Deterministic, no AI cost. Pattern drives the adaptive prompt strategy.

Caching

`CachePort` interface over Redis Lettuce. All operations wrapped in `try/catch` — Redis outage = performance degradation, not service outage. Four cache domains with TTLs from 15 minutes to 24 hours. Cache hit ratio tracked per key prefix (`task`, `topic`, `enrollment`, `template`) via Prometheus counters.

B.15 ADR-014 — Peer Review: Cost-Free Assessment Through Collective Correction

Status: Accepted **Date:** April 2026

Decision

Three-verdict majority vote for `TODO_ACTION` tasks (learner photographs real-world application). Pedagogical basis: reviewing another learner's work is retrieval practice [22]; distributed assessment is more robust than single-evaluator review [20]. Zero marginal AI cost per submission.

Reviewer selection: application-level prioritization (unreviewed → partial → random); not `ORDER BY RANDOM()` (forces full-table scan). Duplicate-reviewer prevention: application filter + database `UNIQUE(submission_assignment_id, reviewer_user_id)`.

Finalization: synchronous on the 3rd verdict (work is bounded — 1 assignment update, at most 4 enrollment updates, milliseconds). No async job needed.

Social feedback: verdict percentage breakdown returned after submission — reviewer sees community consensus after committing their own judgment (anchoring bias prevention by sequencing).

Reviewer reward: 10 points per completed review added to enrollment.

B.16 ADR-015 — Observability Strategy: Custom Metrics, Prometheus, Grafana

Status: Accepted **Date:** April 2026

Decision

Direct `MeterRegistry` injection (not `@Timed` annotations, which cannot capture domain events or set runtime tags). Six instrumented services (Table 4.3).

Key design choices: `Timer.start(registry)` inside the `@Async` body (not the call site — measures generation work, not queue wait); `key_prefix` tag extracted from cache key (`substringBefore(":",")`) — bounded cardinality (4 values); verdicts tracked with `finalized` tag to distinguish raw verdict throughput from submission closures.

Grafana provisioning: datasource and dashboard JSON committed to `infra/grafana/` and loaded automatically on startup — reproducible from `git clone + docker compose up`.

B.17 ADR-016 — Authentication Hardening: Security Audit and Remediation

Status: Accepted **Date:** April 2026

Context

Internal security audit of the Identity bounded context identified 11 gaps (G1–G11) between ADR-011 design intent and actual implementation, discovered by reading every auth file against the ADR specification.

Key Remediations

G1 (Security) Raw magic-link tokens stored in plaintext → fixed: SHA-256 hash stored; raw token only in transit.

G2 (Correctness) TOCTOU race on token claim → fixed: `atomic UPDATE ... WHERE used=false RETURNING * CTE`.

G3 (Security) Google audience validation skipped when `GOOGLE_CLIENT_ID` blank → fixed: blank key = hard startup error.

G5 (Correctness) Refresh rotation discarded email claim → fixed: load `User` by PK during refresh.

G6 (Correctness) Logout only invalidated submitted token → fixed: entire token family revoked.

G7 (Correctness) Public routes returned 401 on stale Bearer token → fixed: filter continues chain on parse failure for public routes.

G8–G10 (Error handling) Auth exceptions, 403, and malformed body rejections returned inconsistent shapes → fixed: `AuthExceptionHandler` returns `MessageResponse` for all error types.

G11 (Operations) No documented path to create first admin → fixed: `V10__seed_initial_admin.sql` Flyway migration, corrected admin promotion SQL in all documentation.

G4 (Deferred) Facebook OAuth audience validation (`/debug_token`) pending `FACEBOOK_APP_ID` configuration.

Outcome

24 unit tests pass (2 new covering family revocation). All auth error responses unified to `MessageResponse`. No breaking API changes — all fixes are server-internal.